

DSA

- <https://github.com/kishanrajput23/Love-Babbar-CPP-DSA-Course/tree/main>
- <https://www.youtube.com/playlist?list=PLDzeHZWIZsTryvtXdMr6rPh4IDexB5NIA>

Table of Contents

- [DSA](#)
 - [Table of Contents](#)
 - [DSA Lecture 1: Flowcharts, Pseudocode & Programming Languages](#)
 - [Table of Contents](#)
 - [Course Intent & Problem Solving](#)
 - [Problem Solving Approach](#)
 - [Development Flow](#)
 - [Flowcharts](#)
 - [Pseudocode](#)
 - [What is Pseudocode?](#)
 - [Example Pseudocodes](#)
 - [Sum of Two Numbers](#)
 - [Even/Odd Check](#)
 - [Sum 1 to N](#)
 - [Prime Number Check](#)
 - [Programming Languages](#)
 - [What are Programming Languages?](#)
 - [Why Do We Need Them?](#)
 - [Language Translation Process](#)
 - [Key Points](#)
 - [Language Rules](#)
 - [Practice Problems](#)
 - [Homework Questions](#)
 - [1. Triangle Validity Check](#)
 - [2. Print Odd Numbers 1 to N \(Inclusive\)](#)
 - [3. Factorial of N](#)
 - [Additional Practice](#)
 - [Mod Operator](#)
 - [Key Takeaways](#)
 - [Problem Solving Steps](#)
 - [Flowchart Components](#)
 - [Programming Fundamentals](#)
 - [DSA Lecture 2: First Program & Data Types](#)
 - [Complete Example Program](#)
 - [Table of Contents](#)
 - [Compilation Process](#)
 - [How Programs Work](#)
 - [Compiler Functions](#)

- [First Program - Hello World](#)
 - [Basic Structure](#)
 - [Code Breakdown](#)
 - [Output Methods](#)
- [Data Types & Variables](#)
 - [Primary Data Types](#)
 - [Variable Declaration Examples](#)
 - [Variable Naming Rules](#)
- [Memory Storage](#)
 - [Integer Storage \(4 bytes = 32 bits\)](#)
 - [Character Storage \(ASCII\)](#)
 - [Negative Number Storage \(2's Complement\)](#)
 - [Signed vs Unsigned](#)
- [Type Casting](#)
 - [Implicit Type Casting](#)
 - [Data Type Conversion Chart](#)
- [Operators](#)
 - [Arithmetic Operators](#)
 - [Division Rules](#)
 - [Relational Operators](#)
 - [Logical Operators](#)
 - [Logical Operators Truth Table](#)
 - [Assignment Operator](#)
- [Key Takeaways](#)
 - [Program Structure](#)
 - [Memory Management](#)
 - [Important Rules](#)

DSA Lecture 1: Flowcharts, Pseudocode & Programming Languages

Table of Contents

- [Course Intent & Problem Solving](#)
- [Flowcharts](#)
- [Pseudocode](#)
- [Programming Languages](#)
- [Practice Problems](#)

Course Intent & Problem Solving

Problem Solving Approach

1. UNDERSTAND THE PROBLEM
↓
2. IDENTIFY GIVEN VALUES
↓

3. THINK ABOUT APPROACH

↓

4. WRITE THE CODE

Development Flow

Problem → Rough Solution → High Level Code → Machine Code → Executable
(Flowchart/
Pseudocode) (C++/Java/
Python) (Binary) (Run Program)

Flowcharts

- [Flowcharts Class Notes](#)

Pseudocode

What is Pseudocode?

- **Pseudo** = Rough, **Code** = Instructions
- Generic way of representing logic
- Language-independent approach
- Easy to understand English-like statements

Example Pseudocodes

Sum of Two Numbers

```
1. Read a, b
2. sum = a + b
3. Print sum
```

Even/Odd Check

```
1. Read N
2. If N%2 == 0
    Then Print "EVEN"
    Else Print "ODD"
```

Sum 1 to N

```
1. Read N
2. sum = 0
3. num = 1
4. While num <= N
    - sum = sum + num
    - num = num + 1
5. Print sum
```

Prime Number Check

```
1. Read N
2. num = 2
3. While num < N
    - If N%num == 0
        Then Print "NOT PRIME" and Exit
    - num = num + 1
4. Print "PRIME"
```

Programming Languages

What are Programming Languages?

Languages used to give instructions to computers, just like we use human languages to communicate.

Why Do We Need Them?

```
Human Instructions → Programming Language → Compiler → Machine Code
"Give me food"      "cout << message"      Binary      Computer Execute
```

Language Translation Process



Key Points

- **Source Code:** Human-readable instructions
- **Compiler:** Translator (Hindi to French analogy)
- **Binary Code:** Machine language (0s and 1s)

- **Executable File:** Ready-to-run program

Language Rules

Every programming language has:

- **Syntax:** Grammar rules
 - **Semantics:** Meaning of constructs
 - **Keywords:** Reserved words
 - **Operators:** Mathematical and logical symbols
-

Practice Problems

Homework Questions

1. Triangle Validity Check

Problem: Check if triangle with sides A, B, C is valid **Logic:** $A+B > C$ AND $B+C > A$ AND $C+A > B$

```
Input: A, B, C
Conditions: All three inequalities must be true
Output: "VALID" or "INVALID"
```

2. Print Odd Numbers 1 to N (Inclusive)

Problem: Print all odd numbers from 1 to N where N is included

```
Input: N = 5
Output: 1, 3, 5
Logic: Start with 1, increment by 2, condition num <= N
```

3. Factorial of N

Problem: Calculate $N! = N \times (N-1) \times (N-2) \times \dots \times 1$

```
Input: N = 5
Output: 120
Logic: fact = 1, multiply by each number from 1 to N
```

Additional Practice

Write pseudocode for all the flowchart examples covered in the lecture.

Mod Operator

- gives remainder
 - $a \% b$ = remainder of a/b
 - $5 \% 2 = 1$
 - $8 \% 2 = 0$
 - $4 \% 9 = 4$, when $a > b$, $a \% b = a$
 - if $n \% 2 = 0$, n is even, else n is odd
-

Key Takeaways

Problem Solving Steps

1. **Understand** the problem statement
2. **Identify** given inputs and required outputs
3. **Design** approach using flowchart/pseudocode
4. **Implement** in programming language

Flowchart Components

- **Oval**: Start/End points
- **Parallelogram**: Input/Output operations
- **Rectangle**: Processing/Calculations
- **Diamond**: Decision making
- **Arrows**: Program flow direction

Programming Fundamentals

- Code needs compilation to run on computer
 - Flowcharts help visualize logic before coding
 - Pseudocode provides language-independent solutions
 - Loops enable repetitive operations
 - Conditions enable decision making
-

DSA Lecture 2: First Program & Data Types

Love Babbar - CodeHelp Series

- [2_first_prof_in_cpp.pdf](#) [IMP] [HowDatatypeStoredInMemory]

Complete Example Program

```
#include <iostream>
using namespace std;

int main() {
    // Variable declarations
    int age = 20;
```

```
char grade = 'A';
bool isPassed = true;
float percentage = 85.5f;

// Output
cout << "Age: " << age << endl;
cout << "Grade: " << grade << endl;
cout << "Passed: " << isPassed << endl;
cout << "Percentage: " << percentage << endl;

// Size information
cout << "Size of int: " << sizeof(int) << " bytes" << endl;
cout << "Size of char: " << sizeof(char) << " bytes" << endl;
// Age: 20
// Grade: A
// Passed: 1
// Percentage: 85.5
// Size of int: 4 bytes
// Size of char: 1 bytes

cout<<"Hello World"
    <<endl
    <<'h'
    <<"\n";
int a=5;
char ch ='a';
bool b=1;
bool b1=true;
float f=1.32;
double d=1.234234234;
cout<<sizeof(a)
    <<endl;
//hi
cout<<(int)'a'<<endl;
cout<<char(65)<<endl;
int aa = 'a';
cout<<aa<<endl;
char chh = 99;
unsigned z=1122;
    unsigned z11=-1122;//wrong way

//comment
/*
comment

comment*/
int a1= 2/5;
cout<<a1;

int a22=2.0/5;
double a2=2.0/5;

int zz =2;
int yy=3;
```

```
bool z1 = (a==b),
z2= (a>b),
z3= (a<b),
z4= (a<=b),
z5= (a>=b),
z6= (a!=b)
;
cout<<endl;
cout<< (true && false)<< (true || false)<< (!true);

cout<<endl;
cout<<(7&4)<<(7|4)<<(7^4)<<(~4)<<(5<<2)<<(16>>2);

int x=10;
float y=x;

float x11=0.14;
int y11= (int)x11;

return 0;
}
```

Table of Contents

- [Compilation Process](#)
- [First Program - Hello World](#)
- [Data Types & Variables](#)
- [Memory Storage](#)
- [Type Casting](#)
- [Operators](#)
- [Key Takeaways](#)

Compilation Process

How Programs Work

```
Source Code → Compiler → Executable File → Output
(.cpp)          (gcc)      (.exe/.out)    (result)
```

- C/ C++ is inbuilt in Linux, as Linux is made in C/ C++ !!!

Compiler Functions

1. **Translation:** Converts source code to machine code

2. Error Detection: Finds compile-time and runtime errors

First Program - Hello World

Basic Structure

```
#include <iostream>
using namespace std;

int main() {
    cout << "NAMASTEY DUNIYA" << endl;
    return 0;
}
```

- IDE
 - Integrated Development Environment
 - Let's You code, debug, execute
 - gives you a lab to experiment
 - eg: VSC(Local IDE), Replit(Remote IDE)
- `;` : end of statement, EOL
- `<<` : means enterline, syntax of cout function
- `return 0;` : end/exit
- `"abcd"` : string & syntax
- library : collection of pre-built code essentials to directly plug and play
- `endl` : newline + flush operation, slower than `\n`
- `iostream` : library contains all input output codes
- `\n` : newline character, faster

Code Breakdown

```
#include <iostream>  ← Include file for input/output functions
using namespace std; ← Use standard namespace
int main() {        ← START block (entry point)
    // code here     ← Program logic
}                   ← Boundary markers
```

Output Methods

```
// Method 1: Using endl
cout << "Hello World" << endl;

// Method 2: Using \n
cout << "Hello World\n";
```

```
// Method 3: Combined
cout << "Hello World" << endl << "Next Line" << endl;
```

Output:

```
Hello World
Next Line
```

Data Types & Variables

- `dataType VarName = Value;`
- `int` = 4bytes, 2bytes etc. DEPENDS ON COMPUTER
- once you defined a Variable with one datatype, YOU CAN'T REDEFINE IN CODE, USE A DIFFERENT VAR !!!
- `char a = '123';` [NO], Can't store more than 1 char
- `char a = "b";` [NO], use single-quotes', not " double-quotes
- even `bool b = true;` then when cout, it gives `1`

Primary Data Types

Data Type	Size	Range	Example
int	4 bytes	-2 ³¹ to 2 ³¹	int a = 5
char	1 byte	0 to 255	char c='a'
bool	1 byte	true/false	bool b=true
float	4 bytes	6-7 digits	float f=1.2
double	8 bytes	15 digits	double d=1.2

Variable Declaration Examples

```
// Integer
int a = 123;
unsigned int ua = 456; // Only positive values

// Character
char ch = 'a'; // Single quotes for char

// Boolean
bool flag = true; // or false, 1, 0
bool isValid = 1;

// Floating point
```

```
float price = 99.99f;
double precision = 3.14159265;

// Finding size
cout << sizeof(a) << endl; // Output: 4
```

Variable Naming Rules

✓ Valid Names:	✗ Invalid Names:
abc1	1abc (starts with number)
_variable	var-name (hyphen not allowed)
myVariable	my variable (spaces not allowed)
MAX_SIZE	#count (special chars not allowed)

Memory Storage

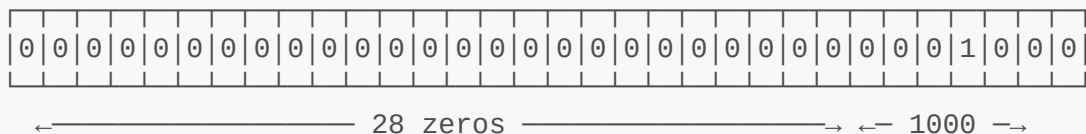
- [2_first_prof_in_cpp.pdf](#) [IMP] [HowDatatypeStoredInMemory]

Integer Storage (4 bytes = 32 bits)

```
int a = 8;
```

Binary of 8: 1000

Memory Layout (32 bits):



Character Storage (ASCII)

```
char ch = 'a';
```

'a' → ASCII value 97 → Binary 01100001

Memory Layout (8 bits):



Negative Number Storage (2's Complement)

First bit = 1 (indicates negative)

Operators

Arithmetic Operators

```
int a = 10, b = 3;

cout << a + b;    // Addition: 13
cout << a - b;    // Subtraction: 7
cout << a * b;    // Multiplication: 30
cout << a / b;    // Division: 3 (integer division)
cout << a % b;    // Modulus: 1
```

Division Rules

```
// Integer / Integer = Integer
int result1 = 2 / 5;    // result1 = 0

// Float / Integer = Float (if stored in float)
float result2 = 2.0 / 5;    // result2 = 0.4
float result3 = 2 / 5;    // result3 = 0 (still integer division)
```

Relational Operators

```
int a = 5, b = 3;

cout << (a == b);    // Equal to: 0 (false)
cout << (a > b);    // Greater than: 1 (true)
cout << (a < b);    // Less than: 0 (false)
cout << (a >= b);    // Greater equal: 1 (true)
cout << (a <= b);    // Less equal: 0 (false)
cout << (a != b);    // Not equal: 1 (true)
```

Logical Operators

```
bool condition1 = true, condition2 = false;

// AND (&&) - All conditions must be true
cout << (condition1 && condition2);    // 0 (false)

// OR (||) - At least one condition must be true
cout << (condition1 || condition2);    // 1 (true)

// NOT (!) - Reverses the condition
```

```
cout << (!condition1);           // 0 (false)
cout << (!condition2);           // 1 (true)
```

Logical Operators Truth Table

A	B	A && B	A B	!A
T	T	T	T	F
T	F	F	T	F
F	T	F	T	T
F	F	F	F	T

Assignment Operator

```
int a = 5;           // Assigns value 5 to variable a
a = 10;              // Updates value of a to 10
```

Key Takeaways

Program Structure

- 1. **#include** - For input/output operations
- 2. **using namespace std** - Use standard namespace
- 3. **int main()** - Entry point of program
- 4. **cout** - For output, **endl** - For newline
- 5. **Semicolon (;)** - Statement terminator

Memory Management

- **int**: 4 bytes, stores integers
- **char**: 1 byte, stores single character (ASCII)
- **bool**: 1 byte, stores true/false
- **float**: 4 bytes, decimal numbers
- **double**: 8 bytes, high precision decimals

Important Rules

- Variable names can't start with numbers
- Integer division gives integer result
- Negative numbers stored using 2's complement
- Type casting happens automatically in mixed operations

